Automatic Question Generation Using Natural Language Processing and Transformers

Reyam Magdy Elshiny
Software Engineering Department
The Briritsh University in Egypt
Cairo, Egypt
Reyam.Magdy@bue.edu.eg

Abeer Hamdy
Software Engineering Department
The Briritsh University in Egypt
Cairo, Egypt
Abeer.Hamdy@bue.edu.eg

2023 International Conference on Computer and Applications (ICCA) | 979-8-3503-0325-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCA59364.2023.10401848

***Abstract*—Online education's rapid growth and the rise of E-learning tools have raised the demand for creating assessments and challenging questions for learners which require significant effort to create suitable content for testing. Consequently, automatic question generation aims to automate the creation of different types of questions in the shortest period of time possible by applying minimal human effort all using natural language processing and transformers. This paper proposes different methodologies to generate questions like true or false, fill in the blanks, matching, multiple-choice, and "Wh-" questions specified from a given context. Transformer models including GPT, T5, and BERT are used to achieve the methodologies needed to generate such questions. The system, tested through surveys, generated an 80% satisfaction rate among teacher participants and showed potential to generate questions similar to ChatGPT's.**

Keywords—transformers, question generation, natural language processing, GPT, BERT, T5

I. Introduction

Rapid global growth necessitates instant production of efficient utilities to meet the diverse community demands and adapt to technological advancements. This rapid growth of technology has impacted the educational sector as well, leading to the use of E-learning and online tools more frequently to enhance teaching quality and student support. Subsequently, testing the learning outcomes of students is essential to assess what students have benefited from throughout their scholarly advancements. Therefore, educators are the ones required to rapidly create and conduct exam questions to evaluate students' understanding of the necessary knowledge. However, educators encounter many challenges that make it difficult to build exam questions on demand to meet a deadline. This is where automatic question generation comes of service to provide an easier approach to generate different types of questions on demand such as true or false, fill in the blanks, matching, multiple-choice, and "Wh-" questions based on a given context using natural language processing and transformer models.

Natural language processing (NLP) is the process of analyzing and interpreting textual data, using computational procedures, found in documents and text files which are called linguistic data. NLP builds a structure from the linguistic data collected which could either be a syntactic or a semantic structure. The syntactic structure describes the grammatical relations among text, while the semantic structure describes the conveyed meaning behind text. As described by Verspoor and Cohen [1], an input text is picked and processed by a pipeline of components to produce the appropriate output text in a natural language processing system.

Transformer models are an advanced branch of natural language processing and neural networks that operate on a sequence of data taken as input and utilizes the input to generate a sequence of output data. Transformers consist of layers of encoders and decoders where encoders manipulate the input data to extract information to be manipulated by the decoder to generate output data. In addition, these models have the ability to understand the relationship between a sequence of data to efficiently predict a target output. Moreover, there are different types of transformer models that manipulate input data, but each can produce different output data which include 1) Bidirectional Encoder Representations from Transformers (BERT) model [2]: an encoder-based transformer language model that is responsible for text recognition, extraction, and sentence classification based on text embeddings. 2) Generative Pre-trained Transformer (GPT) model [2]: a transformer-based language model that is responsible for text generation, completion, and language translation. It is a text generation algorithm that is trained to predict the next word or sentence by capturing the contextual relationships between an input text and the resulting text; it takes an input text and generates a paragraph or more text from that input text. In general, GPT models work on expanding a sentence. 3) Text-To-Text Transfer Transformer (T5) model [2]: a transformer-based language model that is responsible for text summarization, translation, and question answering. T5 combines the encoder and decoder architectures to perform any sequence to sequence (seq2seq) task which makes it stronger than GPT and BERT. T5 is trained to predict missing or corrupted tokens in an input sentence; it rewrites a sentence.

II. Related work

Since the early 2000s, multiple research and projects were dedicated to finding approaches of using NLP for automatic question generation from extracted text. Different types of questions required different algorithms and approaches to achieve them. One of the research projects done by Khokhlov and Reznik [3] proposed approaches to automatically generate questions including fill in the blanks, true or false, and "Wh-" questions using Long Short-Term Memory (LSTM) neural networks. During the question generation process, each sentence is analyzed and categorized into noun, pronoun, and adverb groupings to determine the basis of the question which is done by creating a tree object. Starting with the fill in the blanks question algorithm, it involved four steps. Firstly, a set of predefined questions and gaps were used to train the LSTM model. Secondly, NLP was used to identify verbs, pronouns, and nouns from the tree object then a Stanford parser was used to extract noun and adjective phrases which were then replaced with gaps. Next, stop words were eliminated. Finally, priority was given to numbers and places to be replaced with

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.

gaps. For the true or false question generation algorithm, it involved three methods: 1) Convert every sentence in the list of sentences into false statements except for one. 2) Convert negative words in a sentence into positive ones and vice versa. 3) Manipulate any numbers found in the sentence. Lastly, the "Wh-" question generation involved three steps. Important sentences were first selected. Then the gaps were identified, similar to how they were identified for fill in the blanks question generation and replaced with a "Wh-" word accordingly. The sentence was lastly terminated with "?" mark.

In another article, Goutham [4] proposed an algorithm for automatic true or false question generation from a given context using OpenAI's GPT-2, BERT, and Berkely constituency parsing. In Goutham's algorithm, true statements were directly extracted from the context while generating false statements involved changing the noun or verb phrases from a selected sentence. Generating false statements starts by summarizing the given context. Then, each sentence was split into its noun or verb phrase using constituency parsing. The split sentence was then given as input to GPT-2 completion model to generate a new phrase to complete the split sentence. The GPT-2 model generates a list of false statements which are then filtered using BERT to find the similarity between each of them and the original true statement and give each of the false statements a similarity score. The false statements are then sorted in descending order and the appropriate false statement is selected.

In another research, Lopez et al. [5] proposed a solution of generating paragraph-level questions using transformer models. Their methodology proposed using a dataset called Stanford Question Answering Dataset (SQUAD) [6] which consists of paragraphs each with corresponding question and answer sets. In addition, they used the GPT-2 transformer model which was fine-tuned using six question generation models one of which generated a question for each line in a paragraph which they called One Question Per Line (OQPL) model and the other was All Questions Per Line (AQPL) model. The questions they generated were then evaluated with six evaluation metrics to compare between their results. At last, they discovered that the One Question Per Line (OQPL) model accomplished the highest score.

Later in another study, Rodriguez-Torrealba et al. [7] created a system to generate multiple choice questions (MCQs) using the T5 transformer model. An MCQ question was constructed in five processes. Paragraphs were first extracted from a Wikipedia article and parsed to produce the informative sentences from which the questions will be generated. A question-answer pair was then generated from the produced sentences consisting of a question generated using a set of pre-trained T5 models in addition to its keyword that is the correct answer. Next, a list of distractors that are wrong answers were generated by using the question-answer pair produced along with the previously extracted paragraphs as context. Then, distractors were filtered based on their scores acquired by finding the cosine similarity between the word vectors of each distractor and the correct answer for reference. Lastly, the output of each process was gathered to prepare the MCQs format.

III. Proposed Methodology

Five algorithms are proposed to generate different types of exam questions which include "Wh-" questions, fill in the blanks questions, true or false questions, matching questions, and multiple-choice questions. Each of the approaches proposed begins by receiving an input text which undergoes through a process of tokenization afterwards. Successively, the following sections explain how each algorithm is performed to obtain the required question.

A. "Wh-" Question Generation

"Wh-" questions are questions that ask for something using specific words such as what, where, when, who, and how. Initially, the T5 pretrained transformer model is used and fine-tuned using the SQUAD dataset. After the tokenization process, each word in a sentence is encoded using the T5 encoder which produces a vector token for each word in order for the T5 model to process the sentence as seen in Fig. 1. Each of the tokens have their own input IDs and attention masks [8] which are then obtained to be inserted into the T5 fine-tuned model generator. Once the T5 model is done processing, the relevant question encoding is generated. Lastly, the encoded question is decoded using the T5 decoder which results in the string form of the sentence to become the question. These steps are repeated for each sentence found in the input text.

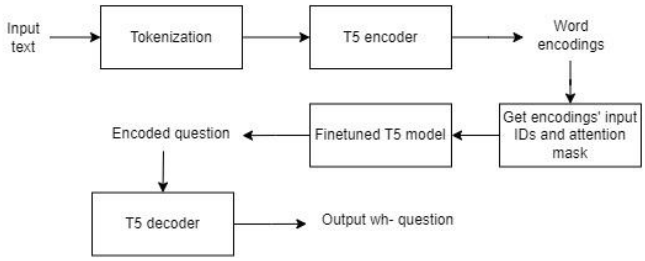


Fig. 1. Wh- Questions Methodology.

B. Fill in the Blanks Question Generation

As an update on Khokhlov and Reznik's [3] work to generate fill in the blanks questions, stop words and punctuation marks are removed from each sentence after the tokenization process which generates the processed sentences as shown in Fig. 2. The processed sentences then go through the PKE keyword extractor which uses unsupervised key phrase extraction with multipartite graphs algorithm [9] to extract specific keywords according to the keywords' parts of speech. A text ranking algorithm is then used to identify the most important nodes (keywords) in the graph according to their weights of the most incoming edges. Nouns, verbs, and adjectives are selected from each sentence to become the candidate keywords after calculating their weights. The important keywords are then returned by PKE to perform further operations on the extracted keywords. A dictionary of keyword-sentences pair is then created by matching and selecting the sentences containing the candidate keywords using the FlashText keyword processor [10]. In the dictionary, each keyword acts as the key to the dictionary and the value of the dictionary is an array of the sentences that contain that specific keyword. Each array of sentences is then sorted in order from the longest sentence length to the shortest. Finally, for each keyword-sentence pair, the longest sentence is selected in order to replace the contained keyword with a blank space using regular expression operations (re) [11] which generates the final output fill in the blanks questions.

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.

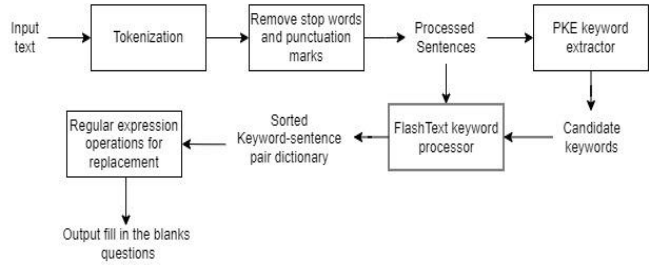


Fig. 2. Fill-in-the-blanks Methodology.

C. Multiple Choice Question Generataion

Generating multiple-choice questions (MCQs) involves three algorithms. The first algorithm is keyword extraction which follows the same steps as with fill in the blanks question generation methodology. The extracted keyword acts as the correct answer in the MCQ. The second algorithm follows the same steps as with "Wh-" question generation methodology using the T5 transformer model. A slight addition is added to the second algorithm where each extracted keyword is forwarded as the answer along with the context to the fine-tuned T5 model instead of forwarding the context only. The third algorithm is the algorithm for generating distractors. Distractors are additional words added as choices that are wrong, yet similar to the domain of the correct answer. For example, if the correct answer to a multiple-choice question is "cat", then the distractors can be "dog", "bird", "hamster", and "fish" which are all under the domain of pets.

Distractors are generated for each of the created questions using the Sense2Vec neural network model [12]. It converts words into vectors in order for the computer to understand them. Using vectors in this case is better since it can find the association between different keywords in the same domain which is needed to generate distractors. As demonstrated in Fig. 3, generating distractors starts by getting the part of speech of the answer keyword using the Sense2Vec "get_best_sense" built-in function which results in the best keyword sense. What is meant by the best keyword sense is the keyword's part of speech in relevance to its position in the sentence. For example, the word "mouse" has two meanings which are both nouns. Depending on the context the word "mouse" is placed in, the meaning or sense behind the word can be inferred. The Sense2Vec "most_similar" built-in function is then used to obtain other keywords that are the most similar to the answer keyword's best sense. The most similar keywords are then filtered by comparing each one with the best sense which returns an array of filtered keywords. The Normalized Levenshtein similarity metric [13] is then used to get the highest similarity score between the answer keyword and each of the filtered keywords. Each score is then compared to a threshold of 0.57 in order for the filtered keywords with the highest similarity score to be appended in the distractors array. Unfortunately, a problem arises where distractors for the same word can be repeated in different forms due to all the word possibilities made available by Sense2Vec. For example, the distractors for the word "cat" with the highest similarity can be "Cat", "cat's", "cats", "Cats", etc. and other distractors such as "dog", "bird", "hamster", "fish", which are the ones targeted, can be at the end of the array. For that reason, the solution is to use sentence transformer model and to find the maximum marginal relevance (MMR) [14] of the distractors to increase the diversity in the output which avoids repetitive options.

The distractors array along with the answer keyword and original processed sentence combined are embedded using the sentence transformer model in order to get the maximum marginal relevance between them. Getting the maximum marginal relevance works by comparing the cosine similarity between each distractor and the answer keyword-sentence combination, and the cosine similarity between the distractors and themselves. Lastly, the top three distractors are selected along with the answer keyword and appended to a dictionary where the generated question is the key to the dictionary. In some cases, a question may not have any distractors to return. In this case the question is removed from the dictionary.

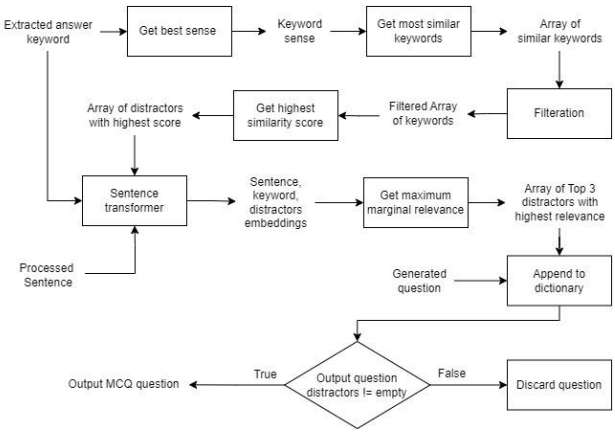


Fig. 3. Multiple-Choice Question Methodology.

D. True or False Question Generation

Generating true or false questions involves three possible modifications to the sentence: (1) the relevant sentence will be left as it is (2) the relevant sentence will be negated (3) the subject phrase will be replaced with another phrase generated using GPT-2 sentence completion model. To generate false statements, the algorithm starts by removing punctuation marks from each sentence which generates the processed sentence after tokenization. As shown in Fig. 4, the processed sentence then goes through the AllenNLP trained constituency parsing neural model [15] which breaks text into sub-phrases (also known as constituents).

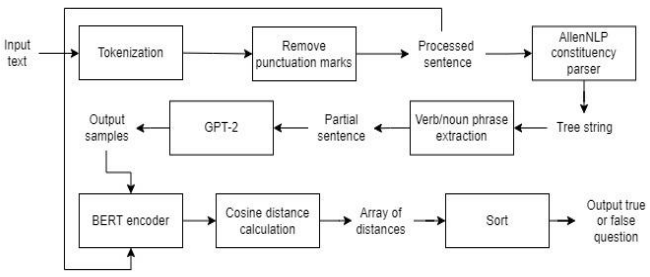


Fig. 4. True or False Methodology.

The NLTK library [16] is then used to create a tree-like structure to visualize the constituency parsing tree string as illustrated in an example in Fig 5 below. Once the constituents are identified, the right most verb phrase (VP) and noun phrase (NP) of the sentence are extracted by going over the constituents of the sentence recursively until the last

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.

verb phrase and noun phrase are approached. The lengths of the verb phrase and noun phrase are then compared to select the longest phrase which is then removed from the original sentence to produce a partial sentence. Similar to Goutham's work [4], the partial sentence then goes through the GPT-2 model to complete the partial sentence with different phrases which creates a set of altered sentences that could be true or false. The generated sentences are stored in a dictionary of output samples with their indices.

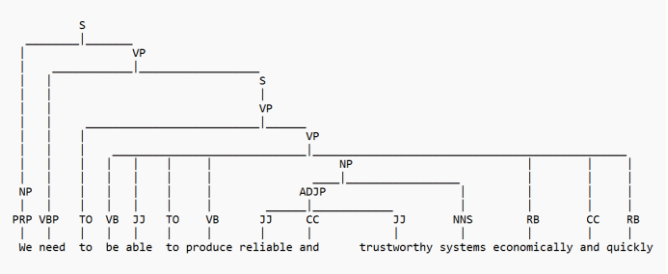


Fig. 5. Constituency Parsing String Tree.

In order to identify which sentence in the output samples is best to be selected to produce a false output, each output sentence is vectorized using the Sentence Transformer BERT model encoder to produce the sentence embeddings. Additionally, the originally processed sentence is also vectorized using BERT encoder in order to then calculate the cosine distance similarity between each sentence from the output samples and the original sentence. The cosine distance calculation results in another array or dictionary holding the distance similarity outcome for each sentence and their indices. The dictionary is then sorted by ascending order of the cosine function's distances. The greater the distance of similarity, the farther the generated sentence is from the original sentence, which is needed to get a false statement. Decoding the sentences back in that case is not needed since the indices provide a guide to where each sentence will be positioned in an array. Lastly, the sentence with the largest distance is selected to become the statement which is added to the true or false exercise. These steps are repeated for each sentence found in the input text to generate false statements.

E. Mathcing Question Gneration

The concept for generating matching questions is to match each keyword extracted from the input text with its corresponding definition. However, a single keyword in a context can have multiple meanings such as the word "mouse" that can mean the rodent animal or the computer mouse, which is considered ambiguous if the word is not used in a context. Therefore, word sense disambiguation (WSD) is used to help identify the best sense for each of the keyword's set of synonyms (abbreviated as synsets) [17]. Generating matching questions follows the same algorithm as fill in the blanks' algorithm for keyword extraction.

As illustrated in Fig. 6, the next step involves using the WordNet library [18] to collect the different synsets for each keyword in the keyword-sentences dictionary.

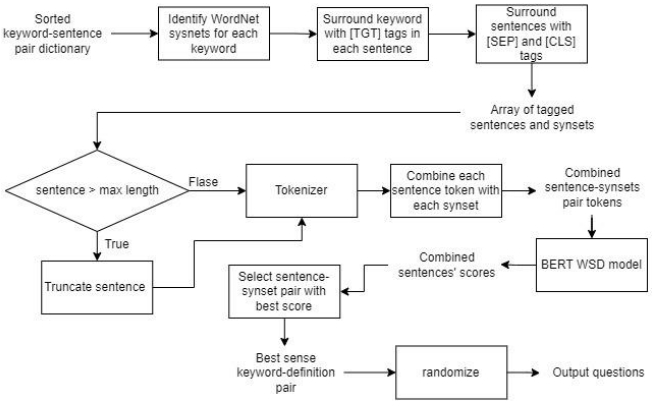


Fig. 6. Matching Methodology.

Additionally, a special "[TGT]" token is placed around the specified keyword in the top three sentences for each keyword-sentence pair in the dictionary which produces an array of tagged words to disambiguate. Moreover, the original sentence is also tagged with the "[CLS]" special token at the beginning of the sentence, and each synset is surrounded by "[SEP]" tokens. This helps differentiate between the different tokens for comparing between the original sentence and each of its synsets as seen in Fig. 7.



Fig. 7. BERT Word Sense Disambiguation [18].

The length of each sentence is then tested to check whether it is greater than the maximum length (which is 125 characters) or not. If the sentence is greater than the maximum length, it goes through the truncation process which shortens the sentence by removing part of original sentence tokens and part of the synset tokens, then goes through the tokenizer step. Otherwise, the sentence continues to the tokenizer step directly. Each tagged sentence along with each of its corresponding sysnets are then tokenized to vectors and combined in order for the BERT word sense disambiguation (WSD) model to process them. The BERT WSD algorithm is trained to generate scores for each sentence-synset pair. The pair that has the highest relevance score is then selected as the best sentence-synset pair for each sentence. Consequently, detokenizing the sentences is not necessary since the selected synset is simply paired with the corresponding keyword in an array. Lastly, the subsequent array is randomized to generate the output matching questions.

IV. Performance Evaluation

The performance evaluation process is held to assess the proposed solutions and algorithms for automating the question generation process. Teaching assistants and some students were asked to evaluate the outcome in a survey where they evaluated different aspects of the generated questions which will be discussed. Additionally, a

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.

comparison between the questions generated by the system and questions generated by OpenAI's ChatGPT system will be conducted. The results of the survey and the comparison will be later discussed in the Results and Discussions section.

A. Survey Evaluation

A survey was conducted to assess the performance and quality of the generated questions by the proposed methods. Each of the assessed question types includes a maximum of 10 generated questions. The participants of the survey included 7 teaching assistants and 3 computer science students from The British University in Egypt. The survey mainly demonstrated a piece of text about Elon Musk [19] from which the exercises were generated. Participants were asked 5 questions in the survey which includes rating the grammatical correctness, relevance, value, coverage, and structure of the generated questions for each exercise on a scale from 1 to 5.

B. Comparison to ChatGPT

OpenAI's ChatGPT is a transformer model which is trained to follow instructions and provide a detailed response to that instruction. It is constructed in a chat or dialogue-like form to answer questions. In order to evaluate the results acquired from the automatically generated questions, ChatGPT is used to compare its results with the generated questions results. Firstly, ChatGPT is supplied with the same context used to generate questions by the Automatic Examination Generator system. Then, ChatGPT was asked to generate "Wh-" questions, multiple-choice questions, fill in the blanks questions, true or false questions, and matching questions.

V. Results and Discussions

A. Survey Evaluation Results

According to the survey conducted, the average results ranged between 3.1 and 4.60 out of 5 as shown below in Fig. 8. The grammatical correctness of the generated questions was rated an average of 4.12 out of 5. While the relevance was rated an average of 3.9 out of 5. Additionally, both the value and coverage of the generated questions were rated 3.84 out of 5. Whereas the structure and organization of the generated questions was rated 4.34 out of 5.

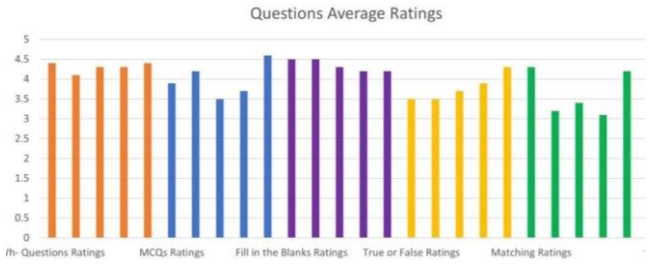


Fig. 8. Average Results for each Question.

In addition, Fig. 9 below shows that the fill in the blanks questions conducted the greatest average satisfaction rate at 86.8% followed by "Wh-" questions with an average rating of 86%. This proves that most participants preferred the generated fill in the blanks and "Wh-" questions over the other types of generated questions. Conversely, matching questions seem to have the least average satisfaction rate which lies at 72.8% followed by true or false questions with an average rating of 75.6%. Finally, the result of the average satisfaction rate of the multiple-choice questions is intermediate which lies at 79.6%. In general, most comments about the generated questions were positive where an overall satisfaction rate of the whole generation gained around 80%, but several participants suggested that the system still needs more improvement to gain better future results.

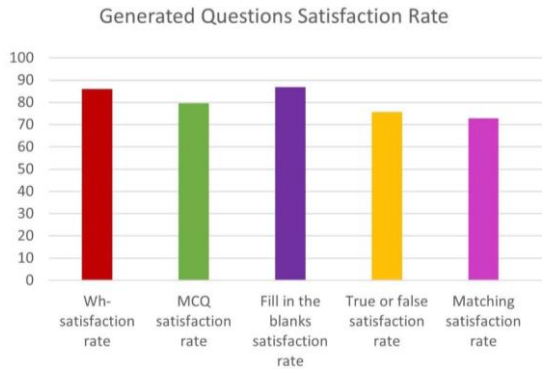


Fig. 9. Survey Results

B. Comparison to ChatGPT Results

The comparison results show that ChatGPT generated better questions since it is more advanced compared to the automatic question generation system. However, some concepts were still covered by the system which was similar to those covered by ChatGPT. For instance, 6 out of 9 of the "Wh-" questions generated by the system were similar in meaning to the "Wh-" questions generated by ChatGPT. What is meant by similar in meaning is that the questions generated by both systems covered the same point from the given context, but each question was stated differently. For example, ChatGPT generated the question "What effect did Elon Musk's tweets have on the cryptocurrency market?" while the Automatic Examination Generator system generated the question "How can Elon Musk influence the digital currency market with just his tweets?" which are quite similar questions. Furthermore, the fill in the blanks questions of both systems were fortunately very similar, the only difference was in the choice of some of the words that were replaced by blanks. Regardless, the automatic question generation system generated fill in the blanks questions where blanks replaced relevant keywords or key phrases to the context.

On the other hand, the multiple-choice questions generated by both systems were completely different. The reason for such difference is that the automatic question generation system generates the question-answer pair based on a single selected keyword, and based on that keyword the distractors are generated similarly whereas ChatGPT generated multiple-choice questions that involved a longer sentence formation of distractors. Moreover, the true or false questions generated by ChatGPT were better and more relevant to the context unlike the true or false questions generated by the automatic question generation system. The reason for this difference is that the system generated true or false questions by truncating part of the sentence and generating a new part using GPT-2 that generalizes the choice of sentence expansion which can sometimes be irrelevant to the context. Additionally, ChatGPT uses GPT-3.5 which is a more advanced model compared to GPT-2. Lastly, the matching questions generated by ChatGPT were better, yet it did not cover all the concepts in the context.

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.

Similarly, the matching questions generated by the automatic question generation system did not cover all concepts as well, but at least three of the selected keywords were similar to those selected by ChatGPT.

Overall, it is clear that ChatGPT generated better output, but the automatic question generation system had some questions that were similar to those generated by ChatGPT such as the fill in the blanks and "Wh-" questions. Subsequently, the automatic question generation system has potential for improvement to generate questions that are as good as ChatGPT's generated questions.

VI. Conclusion

In conclusion, the project proposed solutions to ease, speed up, and automate the process of generating different types of questions to assist educators. Five methodologies were proposed to generate different types of questions for diversity. The five methodologies include "Wh-" questions, multiple-choice questions, fill in the blanks questions, true or false questions, and matching questions methodologies. The "Wh-" question methodology proposes a solution by tuning a T5 transformer model which is trained using the SQUAD dataset consisting of context-question-answer triplets. The fine-tuned model is supplied with the context and generates the appropriate question in return. For the multiple-choice question (MCQ) methodology, three algorithms are proposed. Firstly, the same methodology as "Wh-" question generation is used to generate the question heading of the MCQ. Secondly, a keyword from the context is extracted using the PKE library based on the sentence from which the question is generated; the extracted keyword is the correct answer to the question. Thirdly, distractors are generated using the Sense2Vec model which assigns different distractors from the same domain as the answer keyword. Furthermore, fill in the blanks questions are generated by extracting certain keywords from the context using the PKE library and replacing them with blanks in return. Moreover, the true or false question methodology generates questions by identifying the part of the sentence to be eliminated which is then completed by the GPT-2 transformer model to generate a partially different statement from the original one. Lastly, the matching question methodology involves keyword extraction from the context using the PKE library as well, where a definition for each extracted keyword is obtained from the WordNet dictionary according to its result of the word sense disambiguation performed using the BERT transformer model in order to finally obtain an exercise with randomly shuffled definitions to match the extracted keywords with.

Ultimately, the execution of the system and the generated questions were evaluated to assess the effectiveness of the project. A survey was carried out by participants that included teachers to evaluate the generated questions in addition to carrying out a comparison between the automatic question generation system's and ChatGPT's generated questions. The results showed that around 80% of participants were satisfied with the generated questions, but some suggested that the system needed further improvements to implement in the system. Moreover, the result of comparing the system with ChatGPT showed that the system has a potential of generating questions with the same effectiveness as long as more improvements are carried out.

Nevertheless, the system still has a number of limitations that can be further improved in the future. Some of the questions can be repetitive since all approaches go through each individual sentence to create a question for that sentence. Moreover, the use of GPT-2 to generate true or false questions limits the accuracy and relevance of the false statements. Nonetheless, solutions to those limitations are available and will be assessed to improve the system's performance in the future. With continued enhancement, the system has a great potential of revolutionizing the process of question generation and aiding educators to meet their demands of a dynamic educational environment.

References

[1] K. M. Verspoor, and K. B. Cohen, "Natural language processing," in *Encyclopedia of Systems Biology*, 2013, doi: 10.1007/978-1- 4419-9863-7_158.

[2] M. M. Hasan, "Transformers in natural language processing," 2022, doi: 10.13140/RG.2.2.18062.84809.

[3] I. Khokhlov, and L. Reznik, "Computational intelligence framework for automatic quiz question generation," *Circuits*, 2018, doi: 10.1109/FUZZ-IEEE.2018.8491624.

[4] R. Goutham, "Practical AI: Automatically generate true or false questions from any content with OpenAI GPT2, sentence BERT and Berkley constituency parser," Medium.com, Mar. 2020. [Online]. Available: https://medium.com/swlh/practical-ai-automatically-generate-true-or-false-questions-from-any-content-with-openai-gpt2-9081ffe4d4c9

[5] L. E. Lopez, D. K. Cruz, J. C. B. Cruz, and C. Cheng, "Simplifying paragraph-level question generation via transformer language models," in *PRICAI: Trends in Artificial Intelligence*, pp. 323–334, 2021, doi:10.1007/978-3-030-89363-7_25.

[6] *Stanford Question Answering Dataset*, Hugging Face, 2016. [Online]. Available: https://huggingface.co/datasets/squad

[7] R. Rodriguez-Torrealba, E. Garcia-Lopez, and A. Garcia-Cabot, "End-to-end generation of multiple-choice questions using text-to-text transfer transformer models," *Expert Systems with Applications*, vol. 208, 2022, doi: 10.1016/j.eswa.2022.118258.

[8] A. K. Mehdy, and H. Mehrpouyan, "A multi-input multi-output transformer-based hybrid neural network for multi-class privacy disclosure detection," *Advances in Machine Learning, Data Mining and Computing* [Preprint], 2021, doi:10.5121/csit.2021.111419.

[9] F. Boudin, "Unsupervised keyphrase extraction with multipartite graphs," *North American Chapter of the Association for Computational Linguistics*, 2018, doi: 10.18653/v1/N18-2105.

[10] *FlashText Library*, PyPI, 2018. [Online]. Available: https://pypi.org/project/flashtext/

[11] *Regular Expression Operations (re)*, Python Docs. [Online]. Available: https://docs.python.org/3/library/re.html#id1

[12] Sense2Vec, SpaCy. [Online]. Available: https://spacy.io/universe/project/sense2vec

[13] L. Yujian, L. Bo, "A normailzed levenshtien distance metric," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2007, doi: 10.1109/TPAMI.2007.1078.

[14] Maximum Marginal Relevance. [Online]. Available: https://github.com/MaartenGr/KeyBERT/blob/master/keybert/_mmr.py

[15] AllenNLP, *Allen Institue fro AI*. [Online]. Available: https://allenai.org/allennlp/software/allennlp-library

[16] Natural Language Toolkit. [Online]. Available: https://www.nltk.org/

[17] D. Jurafsky, and J. H. Martin, "Speech and language processing," *Harlow: Pearson Education*, 2021. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/old_sep21/18.pdf

[18] WordNet, Princeton University. [Online]. Available: https://wordnet.princeton.edu/

[19] L. Kolodny, "Elon Musk says Tesla will stop accepting bitcoin for car purchases, citing environmental concerns," *CNBC*, 2021. [Online]. Available: https://www.cnbc.com/2021/05/12/elon-musk-says-tesla-will-stop-accepting-bitcoin-for-car-purchases.html

Authorized licensed use limited to: JSS Science & Technology University. Downloaded on April 11,2026 at 12:25:48 UTC from IEEE Xplore. Restrictions apply.